



Keycloak na Prática

Autenticação e Autorização em Aplicações Modernas

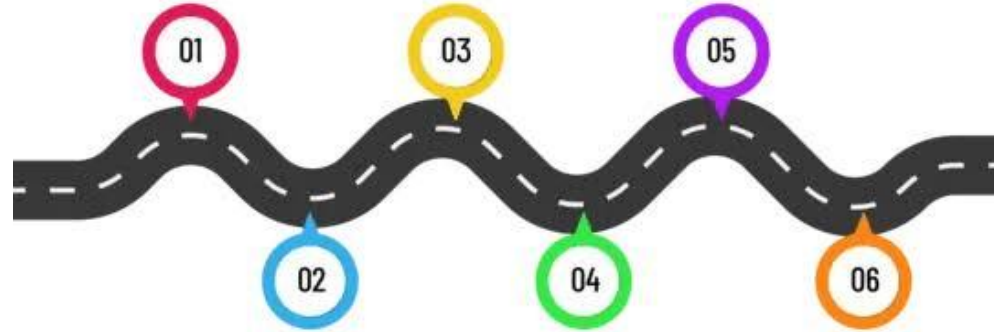


Palestrantes



O que será abordado?

- O que é o Keycloak e onde ele se encaixa.
- Os desafios da autenticação em diferentes aplicações.
- Autenticação, autorização e login único.
- Integração com contas e sistemas externos.
- Protocolos, tokens e fluxos de acesso.
- Organização de usuários, aplicações e permissões.
- Recursos de segurança e personalização.
- Configuração e integração na prática.

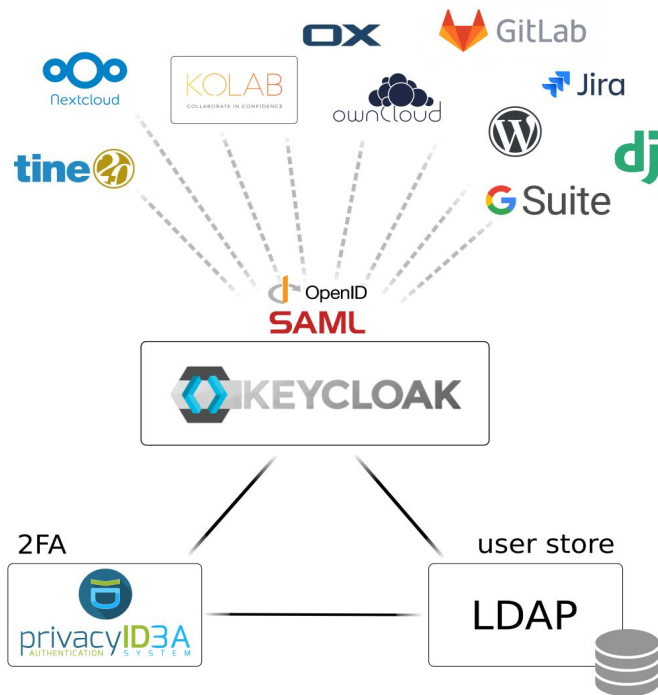


O que é o Keycloak?

O **Keycloak** é uma ferramenta open source de **gerenciamento de identidade e acesso (IAM)** que cuida do **login** e do **gerenciamento de usuários** de aplicações, sem que seja necessário desenvolver toda essa estrutura do zero.

Ele centraliza a autenticação e oferece recursos como:

- **Gerenciamento de usuários e papéis:** administra contas, credenciais, grupos e papéis utilizados no controle de acesso.
- **Login único (SSO):** permite acessar diferentes aplicações a partir de uma única autenticação.
- **Integração com identidades externas:** permite login com provedores como Google e integração com diretórios corporativos, como LDAP e Active Directory.
- **Autenticação multifator (MFA):** aumenta a segurança ao exigir uma confirmação adicional além da senha.



Quem está por trás do Keycloak?

O **Keycloak** surgiu na **Red Hat** e é desenvolvido por uma comunidade de colaboradores, incluindo profissionais da Red Hat e de outras empresas. Também faz parte da **CNCF (Cloud Native Computing Foundation)**, fundação que apoia projetos open source como o Kubernetes.

A **Red Hat** é uma referência em software open source empresarial, com atuação em **RHEL, Fedora, OpenShift, Ansible, JBoss/WildFly e Quarkus**, além de contribuições ao **kernel Linux e ao Kubernetes**. Sua atuação reúne desenvolvimento colaborativo, suporte e manutenção de tecnologias para servidores, aplicações, nuvem e automação.



Alternativas ao Keycloak

Auth0

- Plataforma gerenciada de identidade, da Okta.
- Oferece autenticação, login social, MFA e SSO.
- Facilita a integração sem precisar operar um servidor de identidade.

Amazon Cognito

- Serviço gerenciado de identidade da AWS.
- Oferece cadastro, login, MFA e integração com provedores externos.
- Integra-se ao ecossistema AWS.

Enquanto o Keycloak pode ser hospedado e administrado pela própria equipe, Auth0 e Cognito são serviços operados pelos fornecedores.



Autenticação x Autorização

Autenticação: “Quem é você?”

É o processo de verificar a identidade de quem está acessando o sistema. Pode utilizar senha, chave de segurança ou uma combinação de fatores.

Autorização: “O que você pode fazer?”

É a decisão sobre quais recursos e operações esse usuário pode acessar, considerando seus papéis, permissões e as regras do sistema.

Estar autenticado não significa ter acesso a tudo: cada operação pode exigir uma autorização específica.



Autenticação Tradicional

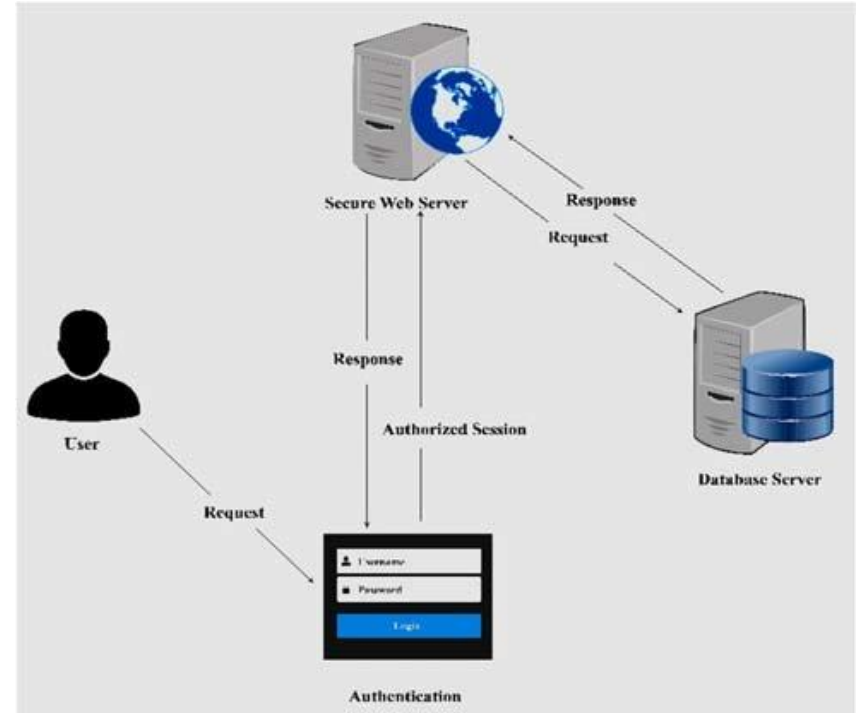
A própria aplicação é responsável por identificar os usuários e gerenciar suas credenciais.

Características

- Login implementado no back-end da aplicação.
- Usuários e hashes das senhas armazenados no banco de dados.
- Sessões ou tokens mantêm o usuário autenticado.

Desafios

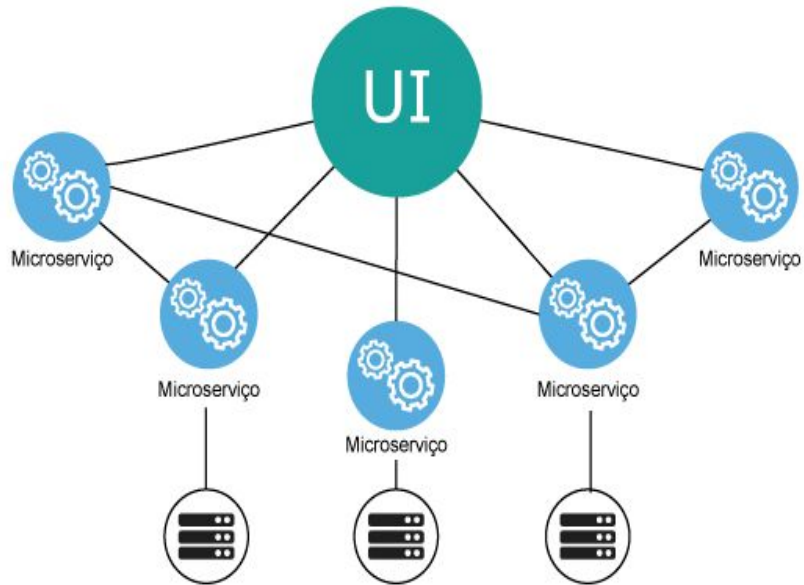
- **Manutenção:** desenvolver cadastro, recuperação de senha e outros recursos de autenticação.
- **Segurança:** proteger credenciais e prevenir ataques ao login.
- **Escalabilidade entre sistemas:** evitar lógica duplicada, contas separadas e políticas diferentes em cada aplicação.



O que mudou?

A autenticação precisa atender a um ecossistema de aplicações, com tecnologias e necessidades diferentes.

- **Múltiplas aplicações:** sistemas desenvolvidos em diferentes linguagens.
- **Diferentes plataformas:** aplicações client-side, server-side, mobile e APIs.
- **Identities de várias fontes:** contas próprias, diretórios corporativos e provedores externos.
- **Experiência integrada:** menos logins repetidos ao alternar entre sistemas.
- **Segurança uniforme:** políticas de autenticação e múltiplos fatores em todas as aplicações.



Uma conta, vários serviços

Uma mesma organização pode oferecer dezenas de serviços:

- **Google:** Gmail, Drive, YouTube e Agenda.
- **gov.br:** Meu INSS, Carteira de Trabalho Digital e outros serviços públicos.
- **Santander:** consultas, pagamentos, cartões e investimentos.

Se cada aplicação exigisse um login independente, o usuário precisaria se autenticar repetidamente.

Como acessar diferentes serviços sem repetir o login?



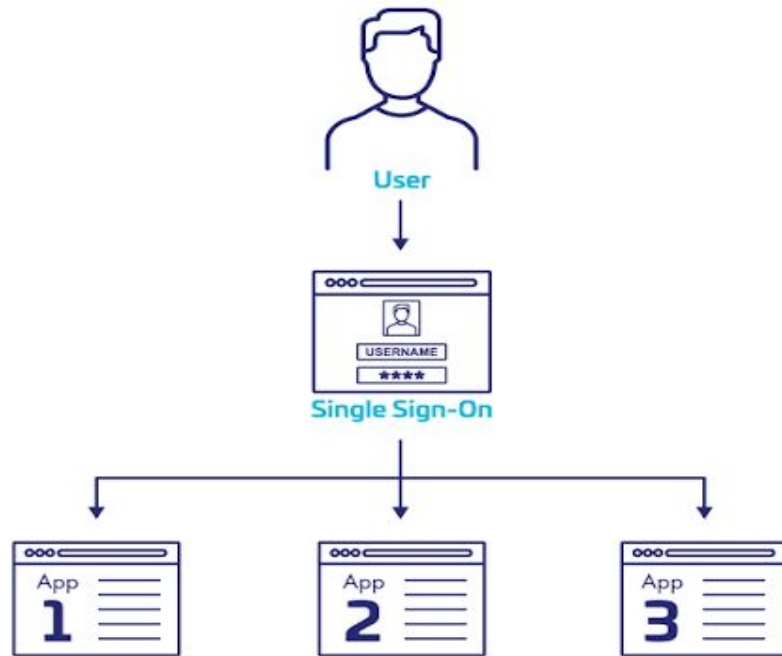
Single Sign On (SSO)

Single Sign-On (SSO) permite que o usuário se autentique uma vez e acesse diferentes aplicações sem informar suas credenciais novamente.

Exemplo: após entrar na conta Google, o usuário pode abrir Gmail, Drive e Agenda aproveitando a sessão existente.

- Mais praticidade: menos interrupções para fazer login.
- Integração: diferentes aplicações confiam em um provedor de identidade.
- Segurança: operações sensíveis ainda podem exigir uma nova confirmação.

O Keycloak oferece SSO para as aplicações integradas a ele.

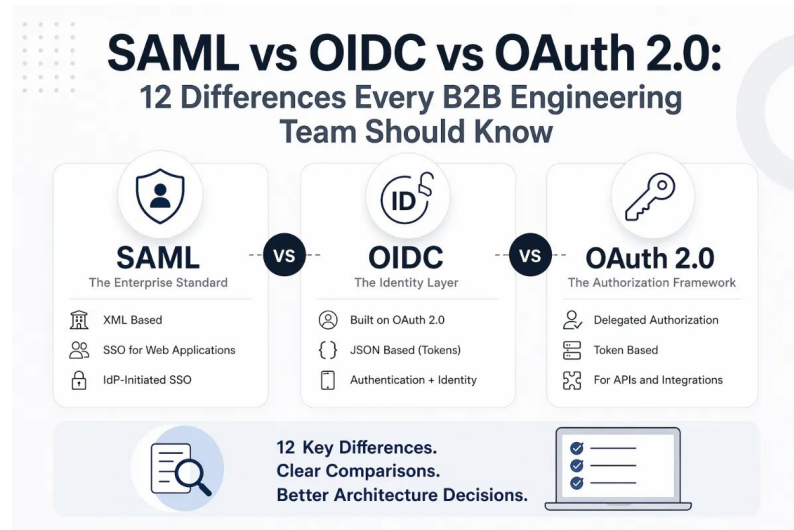


Padrões de integração

O Keycloak utiliza padrões abertos para integrar aplicações desenvolvidas em diferentes tecnologias.

- **OAuth 2.0 (autorização):** permite conceder acesso limitado a recursos, como APIs.
- **OpenID Connect (autenticação):** acrescenta ao OAuth 2.0 a identificação do usuário e informações sobre seu login.
- **SAML 2.0 (autenticação federada):** permite trocar informações de autenticação entre sistemas, sendo muito utilizado em SSO empresarial.

Esses padrões permitem integrar aplicações sem depender de uma linguagem específica.



O que era feito antes?

Para uma aplicação acessar dados de um usuário em outro serviço, era comum pedir que ele fornecesse seu login e sua senha desse serviço.

Exemplo: um aplicativo solicitava a senha do e-mail para importar os contatos do usuário.

Isso trazia problemas:

- Exposição da senha a uma aplicação de terceiros.
- Acesso amplo, sem limitar apenas às informações necessárias.
- Revogação difícil: muitas vezes, era preciso trocar a senha.



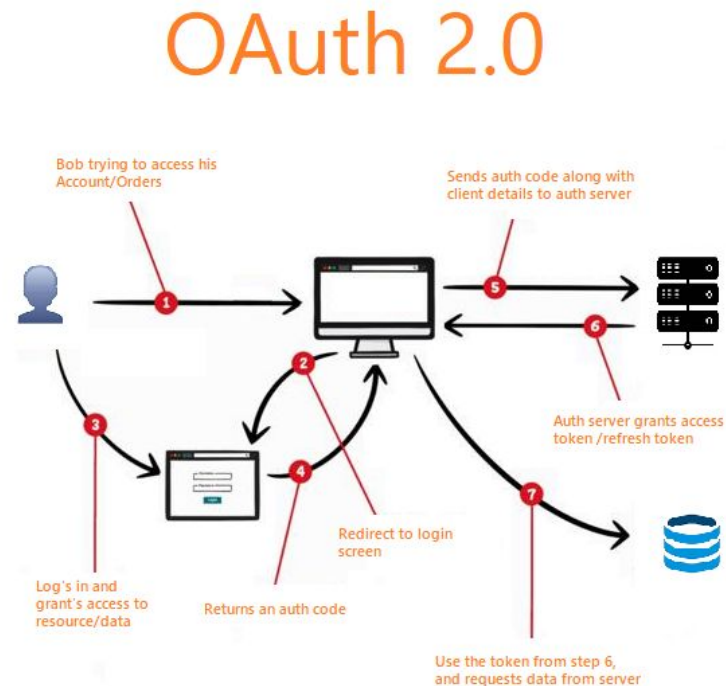
OAuth 2.0

O **OAuth 2.0** é um padrão de **autorização** que permite a uma aplicação acessar recursos de um serviço sem precisar receber a senha do usuário.

Exemplo: um aplicativo quer importar seus contatos do Google.

- Você autoriza o acesso diretamente no Google.
- O aplicativo recebe um access token, com acesso limitado aos recursos autorizados.
- Utiliza esse token para consultar seus contatos.

Você concede acesso aos seus dados, sem entregar sua senha à aplicação.



Papéis do OAuth 2.0

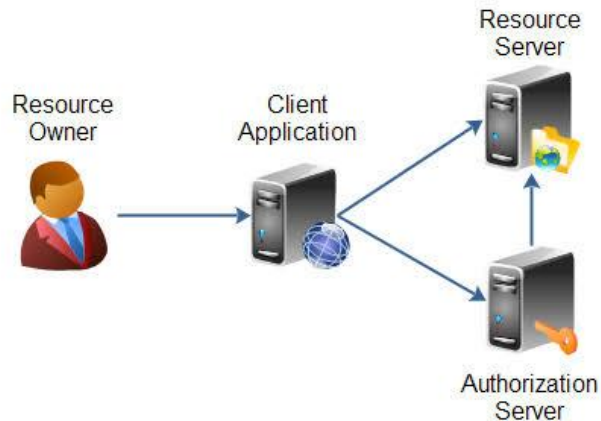
Resource Owner: quem pode autorizar o acesso aos recursos. Geralmente, é o usuário.

Client: aplicação que solicita acesso aos recursos em nome do usuário ou em seu próprio nome.

Authorization Server: servidor que emite tokens após validar a autorização e, quando necessário, autenticar os participantes.

Resource Server: API que disponibiliza os recursos protegidos e valida o access token.

Exemplo: João Pedro (**Resource Owner**) utiliza um aplicativo (**Client**) que obtém um token no Keycloak (**Authorization Server**) para acessar uma API (**Resource Server**).



Tokens

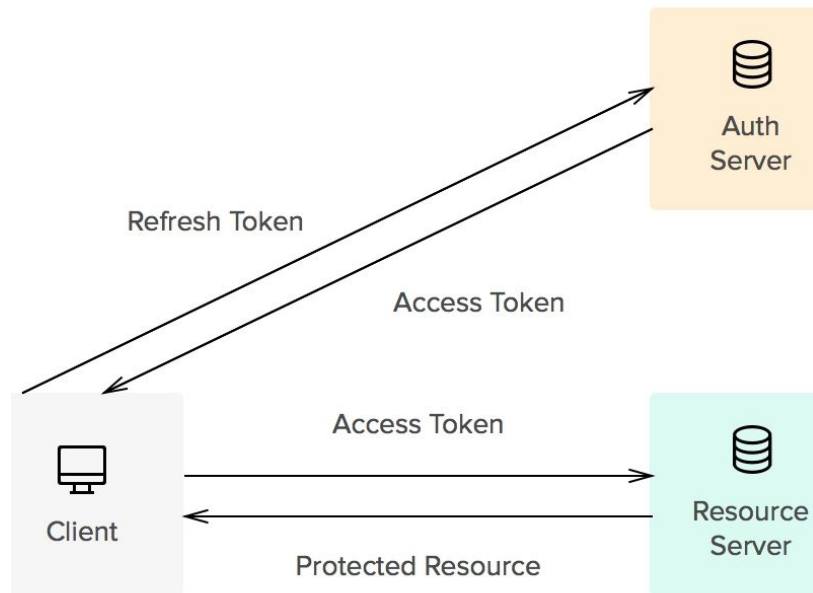
Access token: acesso aos recursos

- É enviado pela aplicação à API.
- Representa o acesso concedido, com permissões e validade determinadas.
- Pode ser um JWT ou utilizar outro formato.

Refresh token: renovação do acesso

- É enviado ao servidor de autorização para solicitar um novo access token.
- Permite renovar o acesso sem exigir uma nova interação do usuário a cada expiração.
- Não é enviado à API e não está presente em todos os fluxos.

Tokens são credenciais: precisam ser protegidos contra exposição.



Grant Types do OAuth 2.0

Grant types são formas de obter um access token, conforme o cenário da aplicação.

- **Client Credentials:** comunicação entre serviços, sem participação de um usuário.
- **Implicit Flow:** fluxo legado para aplicações no navegador.
- **Authorization Code com PKCE:** aplicações web e mobile com participação do usuário.
- **Device Authorization Grant:** dispositivos com entrada limitada, como Smart TVs.

A escolha depende do tipo de aplicação e de quem ela representa: um usuário ou o próprio serviço.



Client Credentials

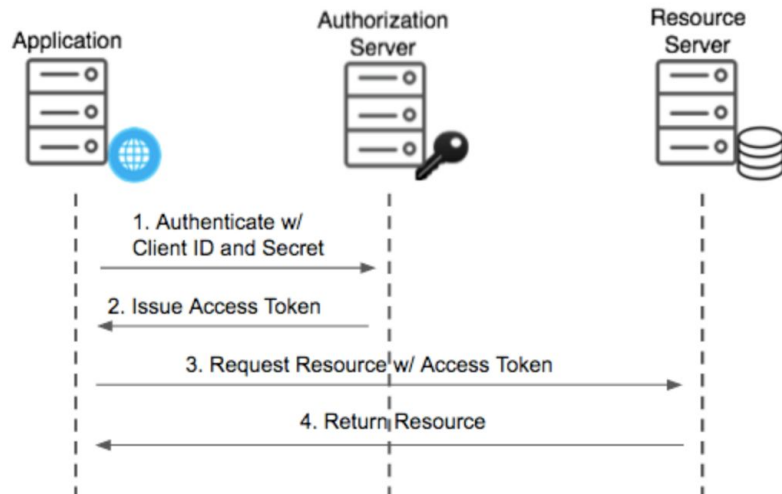
Utilizado na **comunicação entre serviços**, quando uma aplicação acessa uma API em seu próprio nome, sem participação de um usuário.

Como funciona?

1. O serviço se autentica no servidor de autorização, por exemplo, com **client ID** e **client secret**.
2. Recebe um access token com as permissões concedidas.
3. Envia o token nas requisições à API.

Exemplo: um serviço de relatórios consulta uma API de vendas para gerar um relatório diário.

As credenciais devem permanecer no servidor, sem exposição no navegador ou no aplicativo mobile.



Implicit Flow

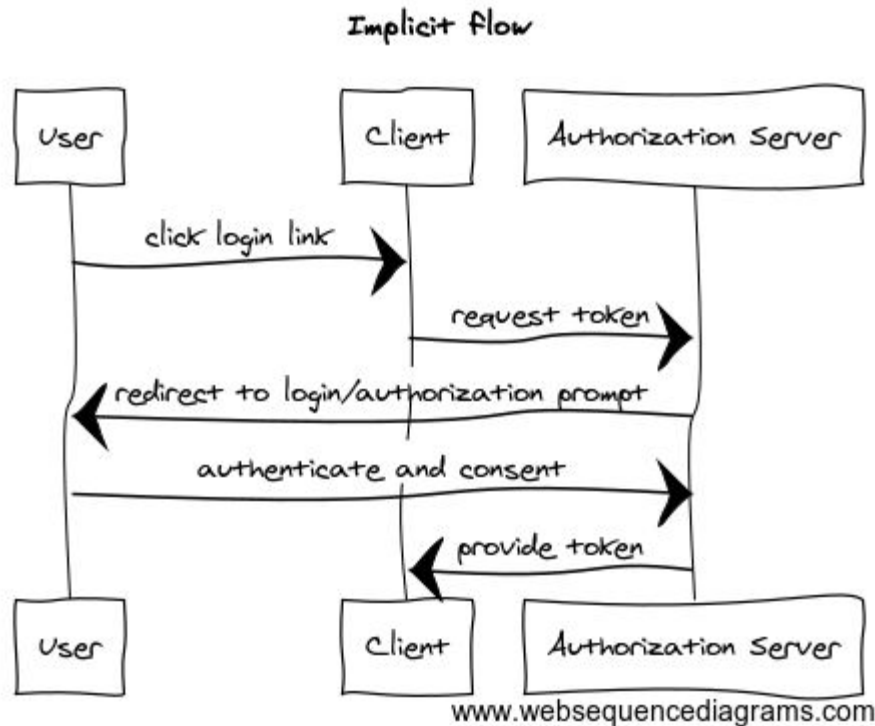
Fluxo criado para **aplicações executadas no navegador**, que recebem o access token diretamente no redirecionamento, sem trocar um código por tokens.

Como funciona?

1. A aplicação redireciona o usuário ao servidor de autorização.
2. O usuário faz login e o acesso é autorizado.
3. O servidor retorna o access token no fragmento da URL.
4. A aplicação extrai o token e o utiliza para acessar a API.

Limitação: o token fica exposto ao ambiente do navegador e pode ser capturado por scripts maliciosos.

É um fluxo legado: para novas aplicações, prefira **Authorization Code com PKCE**.



Authorization Code com PKCE

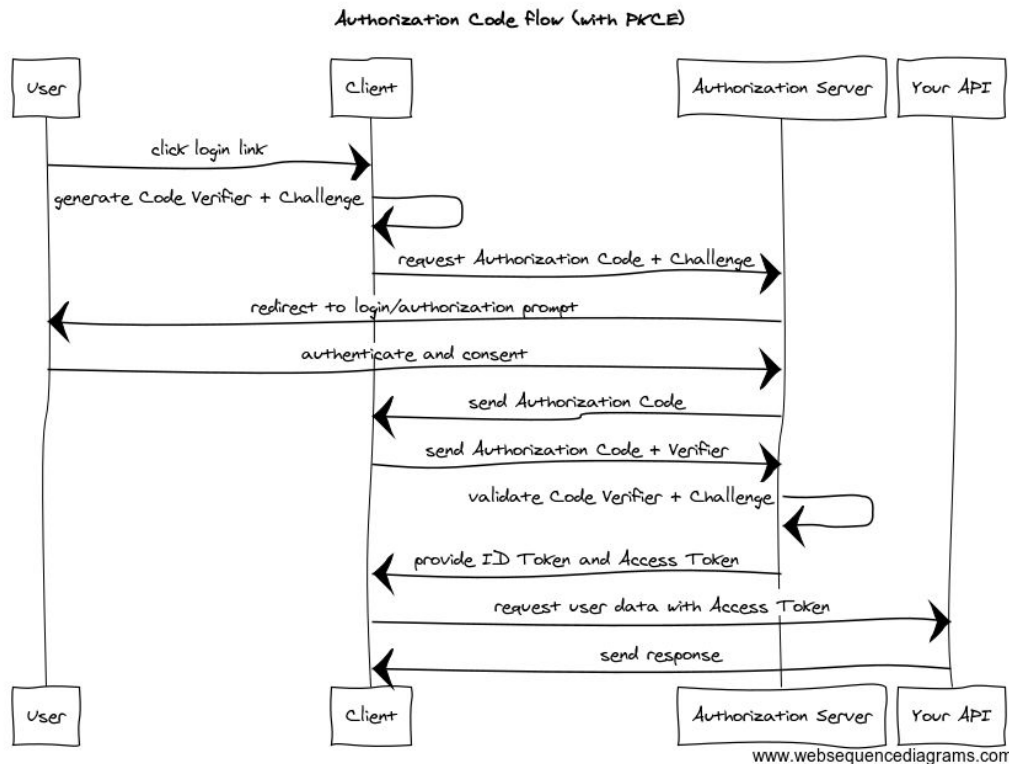
A aplicação recebe um código de autorização e depois o troca por um access token, sem receber a senha do usuário.

Como funciona?

1. A aplicação gera um code verifier e envia seu code challenge ao iniciar a autorização.
2. O usuário faz login no servidor de autorização.
3. A aplicação recebe um código de autorização pelo redirecionamento.
4. Envia o código e o code verifier para obter os tokens.
5. Utiliza o access token para acessar a API.

PKCE: o servidor verifica se o verifier corresponde ao challenge inicial. Assim, interceptar apenas o código não é suficiente para obter os tokens.

Indicado para aplicações web e mobile.



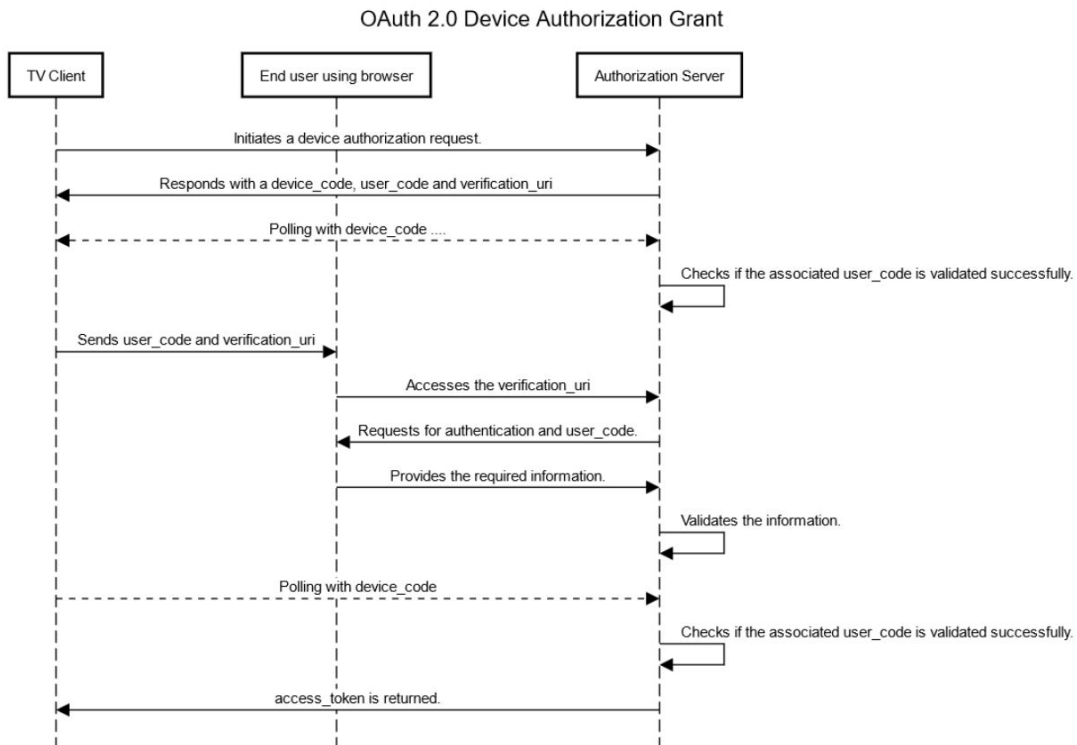
Device Authorization Grant

Utilizado em dispositivos com entrada limitada, como Smart TVs e consoles, permitindo autorizar o acesso pelo celular ou computador.

Como funciona?

1. O dispositivo solicita a autorização e recebe um device code, um user code e um endereço de verificação.
2. Exibe o endereço e o user code ao usuário.
3. O usuário abre o endereço em outro dispositivo, faz login e confirma a autorização.
4. O dispositivo consulta periodicamente o servidor usando o device code.
5. Após a aprovação, recebe um access token para acessar a API.

Exemplo: conectar sua conta a um aplicativo na TV sem digitar a senha pelo controle remoto.



Uma estrutura, vários cenários

Foi justamente o suporte a **diferentes fluxos e cenários de acesso**, combinado ao **controle granular de permissões**, que tornou o OAuth 2.0 tão revolucionário.

- **Versatilidade:** fluxos para aplicações web, mobile, dispositivos e comunicação entre serviços.
- **Permissões granulares:** escopos delimitam o acesso, como consultar dados sem permitir alterações.
- **Acesso temporário:** tokens com validade definida.
- **Interoperabilidade:** integração entre aplicações de diferentes linguagens e tecnologias.

Uma mesma estrutura para conceder o acesso necessário em cada cenário.



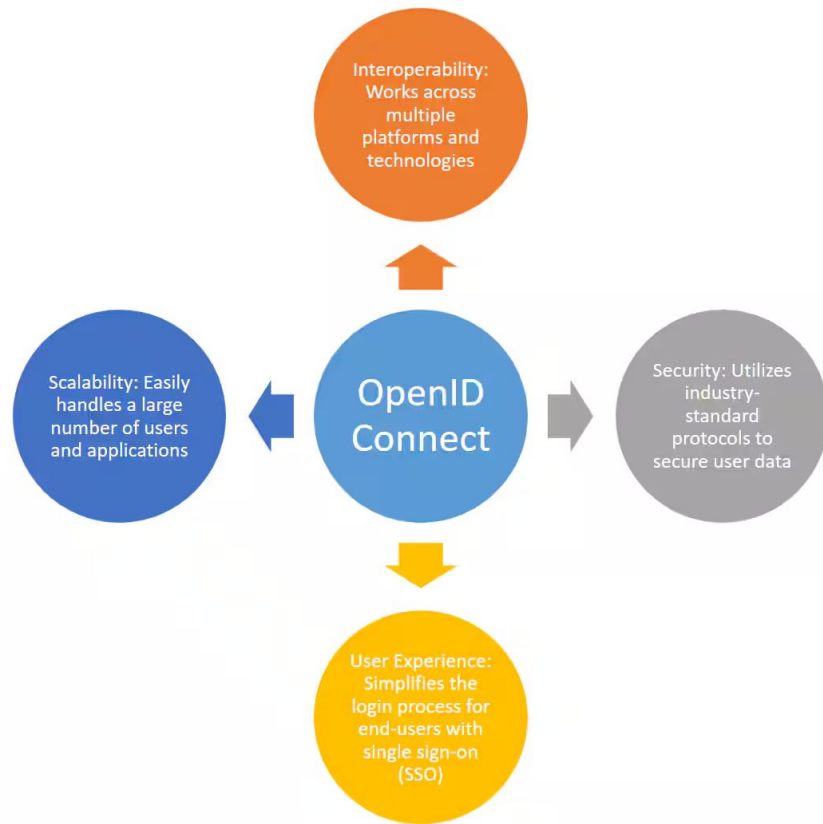
OpenID Connect (OIDC)

O **OpenID Connect** é uma camada de **autenticação construída sobre o OAuth 2.0**. Ele permite que a aplicação verifique quem fez login e obtenha informações sobre esse usuário.

- **ID token:** informa a identidade do usuário e detalhes da autenticação.
- **Claims:** dados presentes no token, como identificador do usuário e, quando disponibilizados, nome e e-mail.
- **UserInfo:** endpoint para consultar informações adicionais do usuário.

Exemplo: após o login no Keycloak, a aplicação valida o ID token para identificar o usuário. Para chamar uma API protegida, utiliza o access token.

OAuth 2.0 permite acessar recursos; **OIDC** permite reconhecer quem está acessando.



OAuth 2.0 x OpenID Connect

OAuth 2.0: Autorização

“Qual acesso esta aplicação recebeu?”

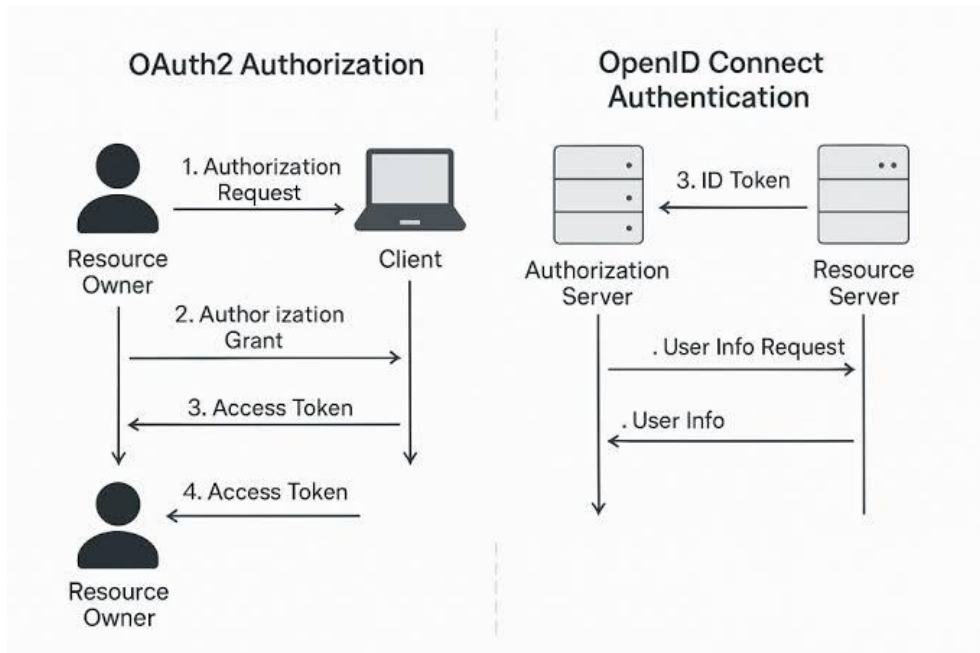
Permite acessar recursos protegidos utilizando um access token.

OpenID Connect: Autenticação

“Quem é o usuário que fez login?”

Adiciona ao OAuth 2.0 um ID token e regras para verificar a identidade do usuário.

Eles trabalham juntos: OIDC identifica o usuário; OAuth 2.0 permite o acesso autorizado às APIs.

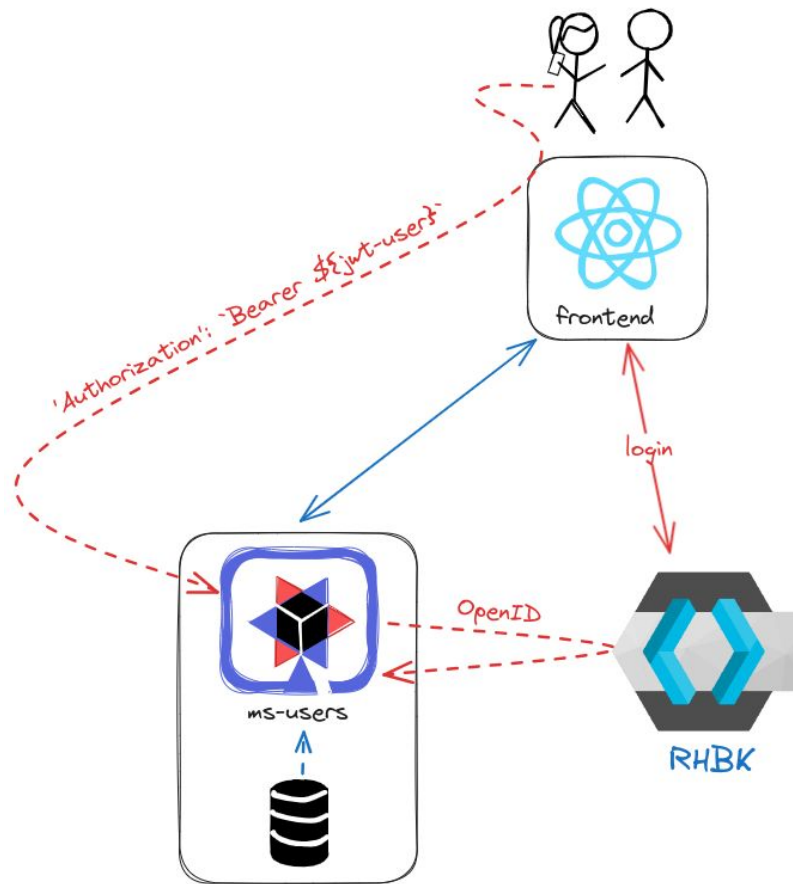


Onde o Keycloak entra nisso?

O **Keycloak** implementa esses padrões e fluxos, fornecendo a estrutura de identidade que as aplicações utilizam.

- **No OAuth 2.0:** atua como Authorization Server, emitindo tokens para acessar APIs.
- **No OpenID Connect:** atua como provedor de identidade, autenticando usuários e emitindo ID tokens.
- **Nos grant types:** oferece os fluxos para aplicações web, mobile, dispositivos e comunicação entre serviços.
- **No SSO:** mantém uma sessão de autenticação que pode ser aproveitada por diferentes aplicações.

As aplicações são os **clients**; as **APIs** são os **resource servers**, que validam os **access tokens** e aplicam as regras de acesso.



Realms

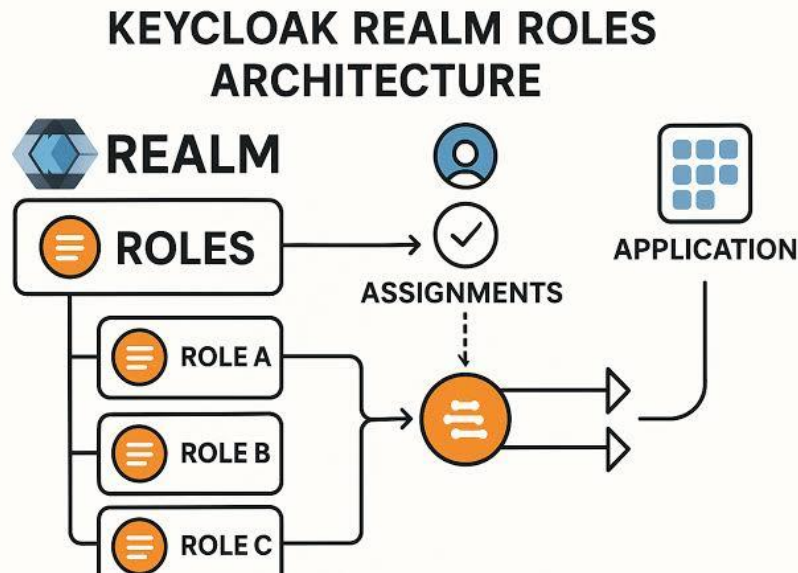
Um realm é um espaço independente que reúne usuários, aplicações e configurações de autenticação.

Cada realm possui seus próprios:

- Usuários, grupos e roles.
- Clients: aplicações e serviços integrados.
- Políticas de senha e fluxos de autenticação.
- Provedores de identidade e fontes de usuários.

Exemplo: o realm faculdade pode reunir o portal acadêmico e o sistema da biblioteca, permitindo que compartilhem usuários e utilizem SSO.

Realms diferentes mantêm contextos de identidade separados.



Clients

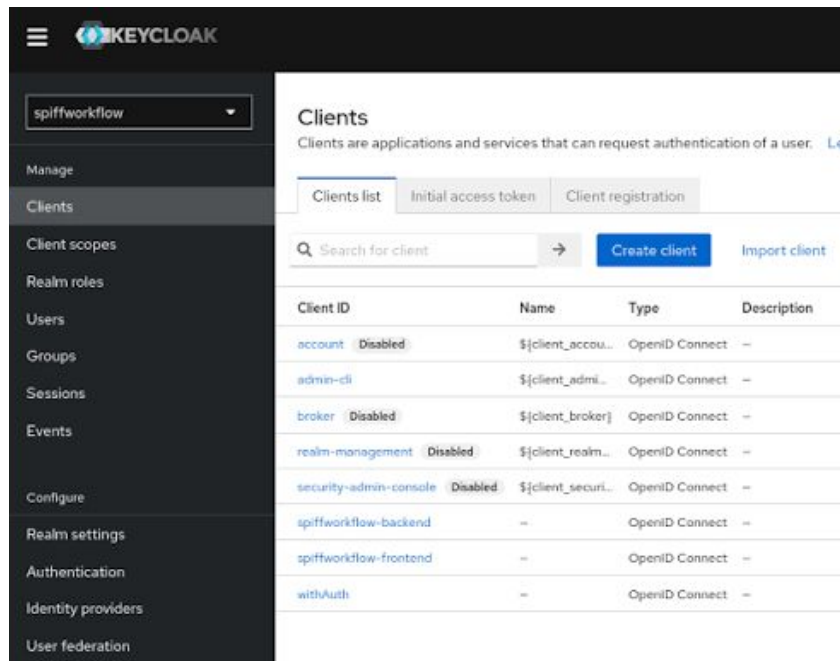
Um client representa uma aplicação ou serviço que utiliza o Keycloak para autenticar usuários ou obter autorização para acessar recursos.

Na configuração do client, definimos:

- **Client ID:** identificador da aplicação.
- **Fluxos habilitados:** como Authorization Code ou Client Credentials.
- **URLs de redirecionamento:** destinos permitidos após o login.
- **Autenticação do client:** se a aplicação precisa comprovar sua identidade, por exemplo, com um secret.

Exemplo: no realm faculdade, o portal acadêmico e o sistema da biblioteca podem ser clients diferentes.

Aplicações no navegador e mobile não conseguem manter um client secret em segurança.



KEYCLOAK

spiffworkflow

Manage

Clients

Client scopes

Realm roles

Users

Groups

Sessions

Events

Configure

Realm settings

Authentication

Identity providers

User federation

Clients

Clients are applications and services that can request authentication of a user. [Learn more](#)

Clients list Initial access token Client registration

Search for client → [Create client](#) [Import client](#)

Client ID	Name	Type	Description
account Disabled	\$_client_accou...	OpenID Connect	—
admin-cli	\$_client_admi...	OpenID Connect	—
broker Disabled	\$_client_broker	OpenID Connect	—
realm-management Disabled	\$_client_realm...	OpenID Connect	—
security-admin-console Disabled	\$_client_securi...	OpenID Connect	—
spiffworkflow-backend	—	OpenID Connect	—
spiffworkflow-frontend	—	OpenID Connect	—
withAuth	—	OpenID Connect	—

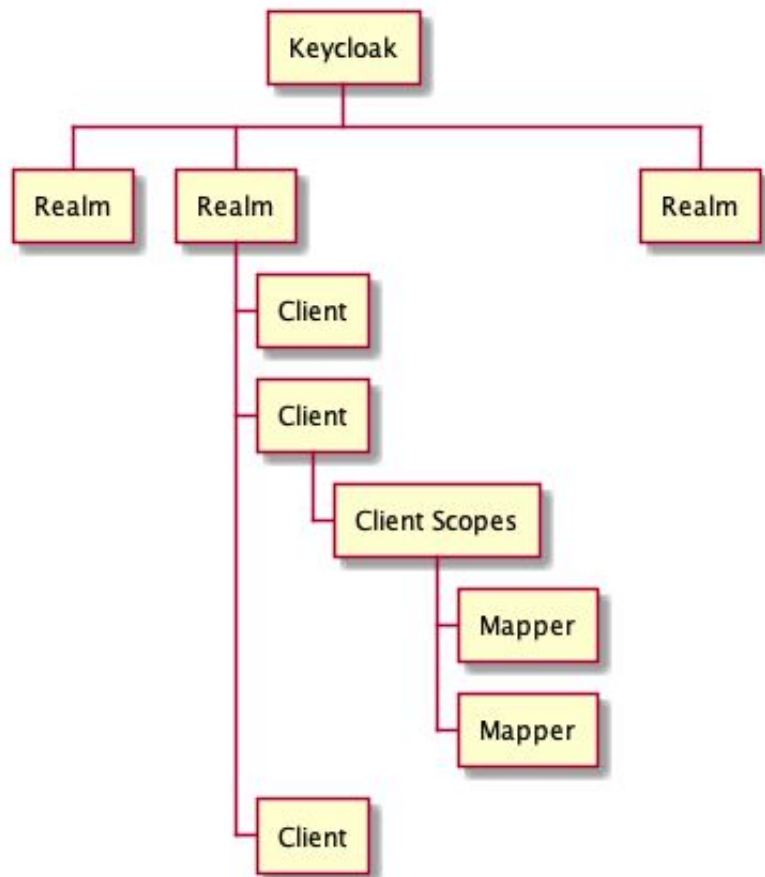
Client Scopes

Client scopes agrupam configurações que definem quais informações e roles podem aparecer nos tokens. Eles podem ser reutilizados por diferentes clients.

- **Protocol mappers:** adicionam claims aos tokens, como nome, e-mail ou atributos personalizados.
- **Role scope mappings:** limitam quais roles do usuário podem ser incluídas.
- **Default scopes:** aplicados automaticamente ao client.
- **Optional scopes:** aplicados quando solicitados pelo parâmetro scope.

Exemplo: o client scope email permite incluir o endereço de e-mail e sua situação de verificação nos tokens, conforme os mappers configurados.

Client scopes configuram o conteúdo dos tokens; a API continua responsável por validar o acesso.

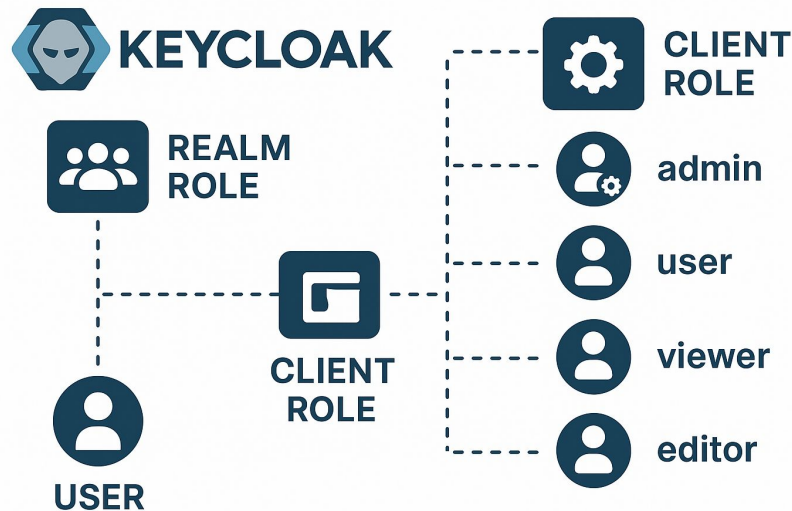


Usuários, grupos e roles

- **Usuários:** representam as pessoas que acessam as aplicações, com seus dados de perfil e credenciais.
- **Grupos:** reúnem usuários com configurações e papéis em comum. As roles atribuídas ao grupo são herdadas pelos seus membros.
- **Roles:** representam papéis, como usuário e admin, que as aplicações utilizam para controlar o acesso.

Exemplo: o grupo administradores recebe a role admin. Ao adicionar um usuário ao grupo, ele herda essa role, sem precisar atribuí-la individualmente.

As roles podem ser definidas para o realm inteiro ou para um client específico.



Integração com identidades externas

O Keycloak pode aproveitar contas que já existem em outros sistemas, evitando um novo cadastro.

Provedores de identidade (IdPs)

- Permitem login com contas do Google, Microsoft ou outro provedor.
- O provedor externo autentica o usuário, e o Keycloak integra esse login às aplicações.



Federação de usuários (User Federations)

- Conecta o Keycloak a diretórios corporativos, como LDAP e Active Directory.
- Permite utilizar as contas que a organização já mantém.



Recursos de Autenticação

Além do login, o Keycloak oferece recursos para proteger e gerenciar o acesso:

- **Autenticação multifator (MFA):** exige uma confirmação adicional, como um código de aplicativo autenticador.
- **Verificação de e-mail:** confirma que o usuário tem acesso ao endereço cadastrado.
- **Recuperação de senha:** permite redefinir a senha por um link enviado por e-mail.
- **Políticas de senha:** definem requisitos como tamanho mínimo e histórico de senhas.
- **Ações obrigatórias:** exigem etapas como atualizar a senha ou configurar MFA antes de concluir o login.



Temas

O Keycloak permite adaptar as telas de login, cadastro e recuperação de senha à identidade visual da aplicação.

Existem diferentes formas de personalizar:

- **Temas prontos:** utilizar projetos disponíveis no GitHub, verificando licença e compatibilidade com a versão do Keycloak.
- **Personalização nativa:** modificar estilos, imagens e templates FreeMarker.
- **Keycloakify:** desenvolver interfaces com React e TypeScript, com testes no Storybook ou em um Keycloak local.

O tema é instalado no servidor e selecionado nas configurações do realm. Com Keycloakify, o build gera um JAR para instalação.

O visual muda; a autenticação continua sob responsabilidade do Keycloak.



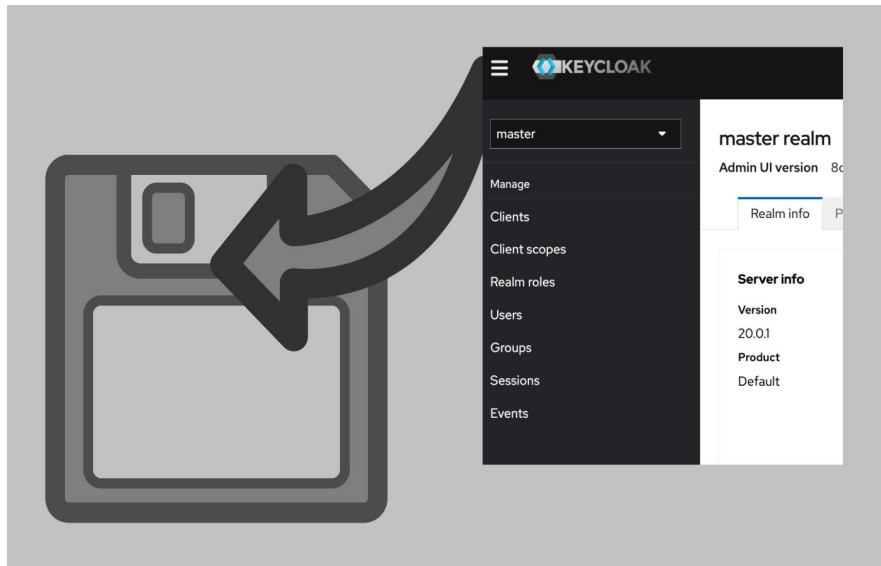
Exportação

O Keycloak permite exportar e importar realms em arquivos JSON, facilitando a reprodução das configurações em outras instalações.

- **Reutilização:** transportar clients, roles, grupos e outras configurações entre ambientes.
- **Versionamento:** acompanhar alterações na configuração utilizando Git.
- **Automação:** importar um realm previamente configurado ao preparar um ambiente.

A exportação pode ser feita pelo painel administrativo, de forma parcial, ou pela linha de comando, com opções mais abrangentes, incluindo usuários.

A exportação não substitui um backup completo do banco de dados.



Cuidados em produção

- **HTTPS:** proteger credenciais e tokens durante a comunicação.
- **Clients bem configurados:** restringir URLs de redirecionamento e habilitar apenas os fluxos necessários.
- **Secrets protegidos:** manter credenciais de serviços no servidor, nunca no front-end ou no código público.
- **Validação na API:** verificar assinatura, emissor, audiência, expiração e permissões dos tokens.
- **Operação confiável:** manter atualizações, backups do banco, monitoramento e um plano de recuperação.

Centralizar a autenticação também torna a disponibilidade do Keycloak essencial para novos logins e renovação de tokens.



Demonstração

Link: <https://github.com/RianErick/keycloak-comsolid>

